GUÍA MAESTRA DE ESTUDIO, ARQUITECTURA Y DEFENSA ORAL

TRABAJO PRÁCTICO N° 3: DESPLIEGUE DE ARQUITECTURA CLOUD-NATIVE

Cátedra: Arquitectura de Sistemas – Licenciatura en Sistemas de Información (Ciclo 2026)

Caso de Estudio: GlobalLogistics S.A. – Plataforma Distribuida de Telemetría y Carga

Equipo de Trabajo: Lucía Astorga, Emmanuel Bustos, Santiago Militello

Repositorio GitHub Oficial: https://github.com/fifix407/tp3-arquitectura-cloud-native-

⚡ RESUMEN EJECUTIVO RÁPIDO (ELEVATOR PITCH DEL TP3)

> **¿De qué se trata este trabajo en 1 minuto?**

> Para la empresa de transporte **GlobalLogistics S.A.**, diseñamos y desplegamos una arquitectura de microservicios en **Docker** automatizada con un solo comando (`docker compose up -d`).

> Cuenta con **1 balanceador de carga Nginx** (Capa 7) que reparte el tráfico en ciclo **Round-Robin** entre **3 réplicas idénticas de una API REST en Python (FastAPI)** dentro de una red virtual privada.

> Cada réplica responde obligatoriamente su **ID de contenedor único** y tiene límites estrictos de hardware (`0.5 CPU` y `512 MB de RAM`).

> Si una réplica se cae, Nginx la saca del clúster automáticamente mediante **Health Checks** y reenvía las peticiones a las réplicas vivas con **cero errores para el usuario (Zero Downtime)**.

> Finalmente, sometimos el sistema a pruebas reales con **Apache Benchmark (100 usuarios concurrentes)**: demostramos que el sistema vuela en consultas normales de telemetría (**561 peticiones por segundo con 39 ms de mediana**), pero colapsa cuando se le exige cálculo matemático pesado de rutas (**la latencia sube a 3 y 6 segundos**), demostrando experimentalmente por qué una arquitectura moderna debe escalar elásticamente con Kubernetes ante picos de CPU.

📌 ÍNDICE GENERAL DEL DOCUMENTO

1. El Problema de Negocio y el Desafío de Ingeniería
2. Topología de la Arquitectura Cloud-Native Diseñada
3. Todo lo que se Construyó en Docker (Paso a Paso Exhaustivo)
4. Infraestructura como Código (docker-compose.yml al detalle)
5. La Capa de Enrutamiento y Balanceo L7 (Nginx y Health Checks)
6. El Microservicio REST (FastAPI, Uvicorn e Inyección de Hostname)
7. Pruebas de Carga Reales con Apache Benchmark (Escenarios 1 y 2)
8. Análisis Crítico Formal y Evaluación de Tiempo Real
9. Banco Extendido de Preguntas y Respuestas para el Coloquio
10. Glosario de Términos Clave para la Defensa Oral
11. Bitácora Completa de Comandos Utilizados y Qué Hace Cada Uno
12. Chaos Engineering: Cómo "Romper" el Sistema, Qué Pasa y Cómo se Mitiga

🏢 1. EL PROBLEMA DE NEGOCIO Y EL DESAFÍO DE INGENIERÍA

1.1 Contexto de "GlobalLogistics S.A."

GlobalLogistics S.A. es un operador logístico internacional que gestiona flotas de transporte terrestre, aéreo y marítimo. Cada camión, barco y centro de distribución envía de forma constante telemetría GPS, confirmación de despachos y solicita recálculos de optimización de rutas de entrega.

1.2 El Problema Previo

La empresa venía operando sobre un sistema monolítico tradicional desplegado en un servidor único. Durante picos comerciales (como CyberMonday o Black Friday), la avalancha de consultas de seguimiento por parte de clientes y los pedidos de optimización de rutas provocaban que el servidor saturara su CPU, encolara las solicitudes y arrojara errores de timeout (504 Gateway Timeout), paralizando las operaciones.

1.3 El Desafío Asignado a Nuestro Equipo

Como Arquitectos Cloud, la dirección técnica nos encomendó diseñar, desplegar y validar un entorno de prueba que demuestre:

1. **Infraestructura como Código (IaC):** Despliegue 100% automatizado, reproducible y sin configuración manual.

2. **Escalabilidad Horizontal:** Un clúster de microservicios con múltiples réplicas idénticas capaces de absorber carga en paralelo.

3. **Alta Disponibilidad y Resiliencia:** Integración de un balanceador de carga que detecte automáticamente cuándo una réplica está enferma o caída (*Health Checks*) y enrute el tráfico sin que el usuario final perciba fallas (*Zero Downtime*).

4. **Validación Experimental de Rendimiento:** Someter la infraestructura a pruebas de estrés con herramientas estándar de la industria (Apache Benchmark) para determinar con precisión matemática sus límites de saturación y su cumplimiento de restricciones de tiempo real.

🏗️ 2. TOPOLOGÍA DE LA ARQUITECTURA CLOUD-NATIVE DISEÑADA

La solución implementa una arquitectura desacoplada en tres capas fundamentales:



Principios Arquitectónicos Aplicados:

- **Separación de Responsabilidades:** Nginx se ocupa exclusivamente de la terminación de conexiones TCP, encolamiento de buffers y distribución de tráfico. Los microservicios solo ejecutan la lógica de negocio.
- **Seguridad Perimetral (Least Privilege Networking):** Ninguna réplica de microservicio tiene puertos expuestos al exterior. Si un atacante intenta escanear los puertos de la máquina host, solo verá el puerto 8080 del balanceador. Los puertos 8000 de las apps son inaccesibles desde fuera de la red virtual de Docker.
- **Apatridia (Statelessness):** Las 3 réplicas son completamente intercambiables. Ninguna guarda estado de sesión en memoria local, lo que permite apagar, reiniciar o añadir réplicas sin afectar la coherencia de la aplicación.

🐳 3. TODO LO QUE SE CONSTRUYÓ EN DOCKER (PASO A PASO EXHAUSTIVO)

Para responder con autoridad si los profesores preguntan *"¿Qué hicieron en Docker?"*, acá tenés el desglose exacto de cada paso técnico realizado en el daemon:

3.1 Creación de la Red Virtual Bridge (`globallogistics_net`)

No usamos la red por defecto (`bridge` default) porque esta no incluye resolución de nombres DNS interna de Docker.

- **Qué se hizo:** Se definió una red dedicada con el driver `bridge`.
- **Beneficio técnico:** Permite que los contenedores se resuelvan entre sí mediante sus nombres de host lógicos (`http://app1:8000`, `http://app2:8000`, `http://app3:8000`), independientemente de la dirección IP efímera que Docker les asigne dinámicamente al iniciar.

3.2 Construcción de la Imagen del Microservicio (`microservice/Dockerfile`)

- **Imagen Base:** Se eligió `python:3.12-slim` en lugar de la imagen de Python completa. La versión *slim* pesa menos de 150 MB frente a los casi 1 GB de la estándar, reduciendo la superficie de vulnerabilidades de seguridad y acelerando los tiempos de build y despliegue.
- **Instalación de Dependencias:** Se copió únicamente `requirements.txt` en un paso previo para aprovechar el sistema de capas de caché de Docker (`Docker Layer Caching`). De esta manera, si se modifica el código en `main.py`, Docker no necesita reinstalar las librerías de Python.

- **Healthcheck Interno:** Se inyectó una instrucción `HEALTHCHECK` nativa de Docker dentro del Dockerfile:

`dockerfile

HEALTHCHECK --interval=10s --timeout=3s --retries=3 --start-period=5s \

CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1

`

Esto permite que el motor de Docker verifique por sí mismo si el microservicio sigue con vida cada 10 segundos.

3.3 Construcción de la Imagen del Balanceador (`nginx/Dockerfile`)

- **Imagen Base:** `nginx:alpine`, la versión más ligera y eficiente del servidor web Nginx (menos de 25 MB).
- **Sobrescritura de Configuración:** Se copió nuestro archivo personalizado `nginx.conf` directamente a `/etc/nginx/nginx.conf`, eliminando la configuración por defecto de Nginx para asegurar que solo responda a los endpoints y reglas de nuestro clúster.

3.4 Contenedor Efímero para Apache Benchmark

Para que las pruebas de estrés fueran 100% reproducibles en cualquier computadora sin necesidad de instalar Apache en Windows:

- Descargamos y corrimos la imagen oficial de Apache HTTP Server en versión Alpine (`httpd:alpine`).
- Ejecutamos el binario `ab` embebido conectándolo a la misma red de Docker (`--network globallogistics_net`).

📜 4. INFRAESTRUCTURA COMO CÓDIGO (docker-compose.yml AL DETALLE)

El archivo `docker-compose.yml` es el cerebro que orquesta todo el despliegue con un único comando. A continuación, el análisis técnico de sus directivas clave:

4.1 Coordinación de Inicio y Dependencias Saludables

```
depends_on:
  app1:
    condition: service_healthy
  app2:
    condition: service_healthy
  app3:
    condition: service_healthy
```

- **Por qué es crucial:** En Docker Compose básico, `depends_on` solo espera a que el contenedor comience a correr, pero el proceso de Python dentro aún puede estar cargando en memoria. Al usar `condition: service_healthy`, Nginx **espera activamente a que las 3 réplicas respondan exitosamente al endpoint `/health`**. Así se elimina el riesgo de que el balanceador envíe peticiones a un contenedor que todavía está arrancando.

4.2 Aislamiento y Cuotas de Recursos (Control de Saturación)

```
deploy:
  resources:
    limits:
      cpus: '0.50'
      memory: 512M
```

- **Mapeo al Kernel de Linux:** Docker utiliza los mecanismos de **cgroups (Control Groups)** y el planificador **CFS (Completely Fair Scheduler)** del kernel. Al asignarle `0.50` CPUs, el kernel le otorga al contenedor una cuota de 50 milisegundos de tiempo de CPU por cada periodo de 100 milisegundos.
- **Justificación en el TP:** Este límite estricto permitió que en el laboratorio pudiéramos inducir saturación de CPU de manera exacta, controlada y sin colapsar la máquina anfitriona.

⚖️ 5. LA CAPA DE ENRUTAMIENTO Y BALANCEO L7 (NGINX Y HEALTH CHECKS)

Nginx opera en la Capa 7 del modelo OSI (Capa de Aplicación), lo que le permite inspeccionar encabezados HTTP, rutas URL y códigos de estado.

5.1 Configuración del Upstream y Health Checks Pasivos

```
upstream microservices_backend {
    server app1:8000 max_fails=3 fail_timeout=10s;
    server app2:8000 max_fails=3 fail_timeout=10s;
    server app3:8000 max_fails=3 fail_timeout=10s;
    keepalive 32;
}
```

- **Algoritmo Round-Robin:** Al no declarar otra directiva como `ip_hash` o `least_conn`, Nginx aplica Round-Robin puro, distribuyendo secuencialmente: Petición 1 ➔ `app1`, Petición 2 ➔ `app2`, Petición 3 ➔ `app3`, Petición 4 ➔ `app1`.
- **Health Checks Pasivos (`max_fails=3 fail_timeout=10s`):** Nginx no bombardea a los servidores con pings continuos innecesarios; en su lugar, monitorea las peticiones reales. Si una réplica falla 3 veces consecutivas, la declara "no disponible" y deja de enviarle tráfico durante 10 segundos. Cumplido ese plazo, le envía una petición de prueba para verificar si ya se recuperó.
- **Reutilización de Conexiones (`keepalive 32`):** Mantiene un pool de 32 sockets TCP abiertos entre Nginx y las réplicas, evitando el costo temporal de abrir y cerrar conexiones TCP en cada request (handshake SYN/ACK).

5.2 Resiliencia y Conmutación por Error (Zero Downtime)

```
proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
proxy_next_upstream_tries 3;
proxy_next_upstream_timeout 5s;
```

- **Mecanismo:** Si Nginx le pasa una solicitud a la Réplica 2 y esta se apaga, se congela o devuelve un error 500, Nginx **no le devuelve el error al cliente**. De forma transparente y en milisegundos, reintenta esa misma solicitud contra la Réplica 3 o 1. El usuario final recibe un código 200 OK y jamás se entera de la falla interna.

5.3 Encabezados de Trazabilidad

```
add_header X-Served-By $upstream_addr always;
add_header X-Upstream-Response-Time $upstream_response_time always;
```

- Cada respuesta HTTP enviada al cliente incluye en sus encabezados la dirección IP interna del contenedor que procesó la orden y los milisegundos exactos que tardó esa réplica en responder.

🐍 6. EL MICROSERVICIO REST (FASTAPI, UVICORN E INYECCIÓN DE HOSTNAME)

El microservicio fue desarrollado en Python 3.12 utilizando **FastAPI** como framework web y **Uvicorn** como servidor ASGI asíncrono.

6.1 Endpoint Principal y Captura Obligatoria del Hostname / Container ID

```
@app.get("/")
async def root(request: Request):
    return {
        "status": "success",
        "service": "GlobalLogistics S.A. - Fleet & Tracking Service",
        "container_id": socket.gethostname(),
        "hostname": socket.gethostname(),
        "instance_name": os.getenv("INSTANCE_NAME", socket.gethostname()),
        "client_ip": request.client.host if request.client else "unknown",
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "architecture": "Cloud-Native Microservices",
        "load_balanced": True
    }
```

- **El truco técnico del Container ID:** La función estándar `socket.gethostname()` consulta la llamada al sistema `uname(2)` del kernel. Dentro de los contenedores Docker en Linux, el motor de Docker asigna como hostname del sistema operativo los primeros 12 caracteres hexadecimales del ID único del contenedor. Así se cumple de manera 100% nativa con la consigna 2.

6.2 Endpoint de Diagnóstico y Salud (`/health`)

```
@app.get("/health")
async def health():
    return {
        "status": "UP",
        "healthy": True,
        "container_id": socket.gethostname(),
        "uptime_seconds": round(time.time() - START_TIME, 2),
        "memory_rss_mb": get_memory_rss_mb()
    }
```

- Permite verificar en tiempo real cuántos segundos lleva encendida la instancia y cuánta memoria física residente (RSS) está consumiendo en megabytes.

6.3 Endpoint de Carga CPU-Intensiva (`/stress`)

```
@app.get("/stress")
async def stress(iterations: Optional[int] = 2500):
    start_calc = time.perf_counter()
    data = f"GlobalLogistics-RouteOptimization-{socket.gethostname()}".encode()
    digest = data
    for _ in range(iterations):
        digest = hashlib.sha256(digest).digest()
    return {
        "status": "processed",
        "container_id": socket.gethostname(),
        "iterations": iterations,
        "duration_seconds": round(time.perf_counter() - start_calc, 4)
    }
```

- Simula una carga de procesamiento pesado (algoritmo de optimización de rutas y criptografía de seguridad) mediante 2.500 rondas encadenadas de cálculo SHA-256 en bucle cerrado. Esto fuerza a la CPU al 100% de uso durante el ciclo.

📈 7. PRUEBAS DE CARGA REALES CON APACHE BENCHMARK (ESCENARIOS 1 Y 2)

Las pruebas se ejecutaron mediante scripts automatizados en Python (`scripts/run_benchmarks.py`) utilizando Apache Benchmark montado en la red virtual de Docker.

7.1 Tabla Comparativa de Resultados Reales Obtenidos

| Atributo / Métrica Temporal | Escenario 1: Tráfico Moderado (I/O) | Escenario 2: Estrés Extremo (Saturación CPU) | Variación e Impacto |

| :--- | :---: | :---: | :---: |

| **Endpoint Evaluado** | `GET /` (Telemetría / Consulta) | `GET /stress?iterations=2500` (Cómputo) | Tipo de carga |

| **Volumen de Peticiones (N)** | 5.000 peticiones | 2.500 peticiones | Volumen calibrado |

| **Nivel de Concurrencia (C)** | 100 usuarios simultáneos | 100 usuarios simultáneos | Paridad de concurrencia |

| **Throughput (Capacidad)** | **561.04 req/segundo** | **170.70 req/segundo** | 🔻 **-69.5% de caída** |

| **Latencia Promedio (Mean)** | **178.24 ms** | **585.83 ms** | 🔺 **+228.7% de lentitud** |

| **Mediana (Percentil 50 - p50)** | **39 ms** | **177 ms** | 🔺 **+353.8% de lentitud** |

| **Percentil 75 (p75)** | **94 ms** | **396 ms** | Inicio de degradación |

| **Percentil 95 (p95)** | **966 ms** | **3.016 ms (3.01 seg)** | 🔺 **3 veces más lento** |

| **Percentil 99 (p99)** | **2.713 ms** | **6.258 ms (6.26 seg)** | 🔺 **Cola larga crítica** |

| **Tasa de Éxito HTTP (2xx)** | **100.00% (0 errores)** | **92.52% (éxito en la mayoría)** | Colas en buffers |

7.2 Interpretación de los Dos Escenarios

1. **Escenario 1 (Régimen Elástico):**

* El 50% de los usuarios recibió su respuesta en apenas **39 milisegundos**.

* El 75% tardó menos de **94 milisegundos**.

* Las tres réplicas absorbieron holgadamente más de 560 pedidos por segundo sin fallar un solo paquete de red.

2. **Escenario 2 (Régimen de Saturación de CPU):**

* Al exigir cálculo intensivo, la capacidad de procesamiento se desploma a 170 req/s.

* La mediana sube a 177 ms, pero a partir del p75 la curva se vuelve exponencial: el percentil 95 llega a **3 segundos** y el 99 a **más de 6 segundos**.

🧠 8. ANÁLISIS CRÍTICO FORMAL Y EVALUACIÓN DE TIEMPO REAL

Esta es la parte teórica que más valoran los docentes en la materia de Arquitectura de Sistemas:

8.1 ¿En qué punto exacto se degrada el sistema?

El punto de degradación se produce en la **transición de operaciones I/O-bound hacia operaciones CPU-bound**:

- Cuando las peticiones son de lectura simple, el cuello de botella es la red y el event loop asíncrono de FastAPI las procesa en milisegundos sin bloquear hilos.
- Cuando las peticiones requieren cálculo matemático pesado (SHA-256), monopolizan el hilo principal de Python. Como tenemos 100 clientes concurrentes compitiendo por un clúster acotado a 1.5 núcleos de CPU (0.5 cores x 3 réplicas), el planificador del kernel Linux suspende las réplicas que agotan su cuota de 50 ms por ciclo (*CFS Throttling*). Las peticiones comienzan a encolarse en el buffer del socket TCP, provocando un retraso acumulativo exponencial.

8.2 ¿Cumple con Restricciones de Tiempo Real?

En la ingeniería de software existen dos grandes tipos de sistemas de tiempo real:

1. **Sistemas de Tiempo Real Estricto (Hard Real-Time):**

* Son sistemas donde incumplir un plazo de entrega (*deadline*, por ejemplo tardar más de 50 ms) es considerado una falla catastrófica (ej. frenos ABS de autos, marcapasos o control aéreo).

* **Conclusión:** Nuestra arquitectura **NO CUMPLE bajo ninguna circunstancia**. A pesar de que la mediana en el Escenario 1 es de 39 ms, la varianza introducida por el sistema operativo y la virtualización genera que el percentil 99 llegue a 2.7 segundos.

2. **Sistemas de Tiempo Real Blando (Soft Real-Time / Web Transaccional):**

* Son sistemas donde un retraso degrada la calidad de la experiencia pero no invalida los datos ni destruye el sistema (ej. seguimiento de envíos de paquetes).

* **Conclusión en Tráfico Moderado: CUMPLE** con los acuerdos de nivel de servicio comerciales estándar (SLA de p95 < 1.000 ms, habiéndose registrado 966 ms).

* **Conclusión en Saturación de CPU: NO CUMPLE**, ya que latencias de 3 a 6 segundos provocarían que los usuarios abandonen la página o que las aplicaciones móviles disparen timeouts de red.

8.3 Justificación Arquitectónica del Comportamiento

- **Limitación del GIL y Concurrencia Monohilo:** CPython posee un *Global Interpreter Lock (GIL)*. Aunque FastAPI es asíncrono, las tareas de cálculo pesado en un solo proceso bloquean el bucle de eventos. Para tareas CPU-bound en producción, se requeriría configurar múltiples workers por réplica (`--workers 2` o más) o derivar el trabajo a colas de tareas en segundo plano (Celery / RabbitMQ / Redis).
- **Comportamiento del Algoritmo Round-Robin:** Round-Robin es ciego al estado interno de los contenedores. Si la Réplica 1 está procesando una ruta que tarda 500 ms y la Réplica 2 terminó su tarea en 10 ms, Round-Robin seguirá

enviando solicitudes a la Réplica 1 en su turno. Un algoritmo `least_conn` habría redirigido el tráfico hacia la réplica con menos conexiones activas.

8.4 Solución Cloud-Native Recomendada

Para solucionar el problema observado en el Escenario 2 sin intervención humana, la arquitectura debe evolucionar hacia un orquestador de contenedores como **Kubernetes**:

- Configurar un **HPA (Horizontal Pod Autoscaler)** basado en la métrica `targetCPUUtilizationPercentage: 70%`.
- Cuando el clúster detecte que las 3 réplicas superan el 70% de uso de procesador, el autoscaler instanciará automáticamente réplicas adicionales (escalando de 3 a 10 pods) y Nginx distribuirá la carga entre más núcleos de cálculo.

❓ 9. BANCO EXTENDIDO DE PREGUNTAS Y RESPUESTAS PARA EL COLOQUIO

Pregunta 1: "¿Por qué el docker-compose.yml tiene exactamente 3 réplicas y no usaron la directiva `scale`?"

> **Respuesta del alumno:**

> "Definimos explícitamente tres servicios (`app1`, `app2`, `app3`) para poder configurarles de forma determinística nombres de contenedor fijos (`globallogistics_app_1`, etc.), variables de entorno individuales (`INSTANCE_NAME`) y dependencias estrictas de salud (`service_healthy`) con Nginx. Si bien `docker compose up --scale app=3` es válido para escalado dinámico, definir las instancias explícitamente en el manifiesto nos garantizó asignarles nombres fijos en el bloque `upstream` de Nginx y límites de CPU individuales exactos de 0.5 cores a cada una."

Pregunta 2: "¿Cómo comprobaron que el balanceador realmente estaba alternando entre instancias?"

> **Respuesta del alumno:**

> "Desarrollamos un script automatizado en PowerShell (`scripts/verify_loadbalancing.ps1`) que ejecutó 15 peticiones consecutivas al puerto 8080. Capturamos los campos `container_id` e `instance_name` de cada respuesta HTTP y generamos una tabla donde se observa una alternancia cíclica perfecta: 5 solicitudes fueron atendidas por la réplica 1, 5 por la réplica 2 y 5 por la réplica 3, confirmando una distribución simétrica exacta del 33.3%."

Pregunta 3: "¿Qué sucede si una réplica se cae en mitad de una operación? ¿El usuario ve un error?"

> **Respuesta del alumno:**

> "No, el usuario no ve ningún error gracias a la directiva `proxy_next_upstream` que configuramos en `nginx.conf`. Si la réplica que tenía asignada la petición se cae o devuelve error 502/504, Nginx intercepta ese fallo y reenvía inmediatamente la petición a otra réplica viva del pool antes de cerrar la conexión con el cliente. Hicimos la prueba apagando manualmente el contenedor `globallogistics_app_2` con `docker stop` y el sistema continuó funcionando con cero tiempo de inactividad (Zero Downtime)."

Pregunta 4: "¿Por qué es un error guiarse únicamente por la latencia promedio?"

> **Respuesta del alumno:**

> "Porque el promedio es una métrica engañosa que esconde la mala experiencia de los usuarios en la cola larga. Por ejemplo, en nuestro Escenario 2 el promedio dio 585 ms, lo que parecería aceptable. Sin embargo, al analizar los percentiles descubrimos que el percentil 95 sufrió una espera de 3.016 ms (más de 3 segundos) y el percentil 99 superó los 6.2 segundos. En arquitecturas modernas y contratos de SLA, la calidad del servicio se mide en p95 y p99 para proteger a los usuarios más desfavorecidos."

Pregunta 5: "¿Por qué las réplicas no exponen puertos hacia Windows en el archivo Compose?"

> **Respuesta del alumno:**

> "Por el principio de seguridad de Mínimo Privilegio y Aislamiento de Red. El único punto de entrada público de la arquitectura debe ser el balanceador de carga en el puerto 8080. Las réplicas se comunican internamente a través de la red virtual privada `globallogistics_net`. Si publicáramos los puertos de las réplicas en `0.0.0.0:8000`, cualquier cliente externo podría eludir el balanceador de carga y saturar una réplica específica de forma directa."

Pregunta 6: "¿Qué diferencia existe entre un Health Check Activo y uno Pasivo?"

> **Respuesta del alumno:**

> "Un Health Check Activo envía periódicamente solicitudes sintéticas a un endpoint (`/health`) para consultar si el servicio está vivo (es lo que hace Docker con la instrucción `HEALTHCHECK` cada 10 segundos). Un Health Check Pasivo, en cambio, no genera tráfico adicional; monitorea el tráfico real de los clientes. Nginx utiliza Health Checks pasivos con `max_fails=3`: si detecta que 3 peticiones reales de usuarios fallan contra una réplica, concluye que está enferma y la suspende temporalmente durante 10 segundos."

📖 10. GLOSARIO DE TÉRMINOS CLAVE PARA LA DEFENSA ORAL

- **IaC (Infraestructura como Código):** Práctica de gestionar y aprovisionar infraestructura informática a través de archivos de definición legibles por máquina (como `docker-compose.yml`), en lugar de configuraciones físicas o herramientas interactivas manuales.
- **Reverse Proxy (Proxy Inverso):** Servidor intermedio que se sitúa frente a uno o más servidores web, interceptando las solicitudes entrantes de los clientes y reenviándolas al servidor de backend correspondiente.
- **Round-Robin:** Algoritmo de planificación y balanceo que distribuye las solicitudes a través de una lista de servidores en orden circular secuencial.
- **Throughput (RPS):** Cantidad de peticiones completadas con éxito por unidad de tiempo, típicamente expresada en *Requests Per Second (req/s)*.
- **Percentil 95 (p95):** Umbral temporal en el cual el 95% de las solicitudes se completaron en un tiempo menor o igual a dicho valor, y solo el 5% de las solicitudes fue más lenta.
- **CFS Throttling:** Mecanismo del kernel Linux por el cual un proceso dentro de un contenedor es suspendido temporalmente por haber consumido la cuota de CPU asignada dentro de su periodo de tiempo.
- **SLA (Service Level Agreement):** Compromiso formal entre un proveedor de servicios y un cliente sobre métricas mínimas de rendimiento y disponibilidad (ej. p95 < 1 segundo y disponibilidad > 99.9%).

⚙️ 11. BITÁCORA COMPLETA DE COMANDOS UTILIZADOS Y QUÉ HACE CADA UNO

Si los profesores preguntan *"¿Qué comandos usaron para armar, probar y monitorear esto?"*, acá tenés la lista completa con su explicación técnica:

11.1 Gestión y Orquestación del Clúster (Docker Compose)

- `docker compose up -d --build`

* **Qué hace:** Compila los dos Dockerfiles (`microservice` y `nginx`), crea la red virtual `globallogistics_net` , levanta los 4 contenedores en segundo plano (`-d`) e inicia la supervisión de Health Checks.

- `docker compose ps`

* **Qué hace:** Muestra el estado del clúster de Compose (si están corriendo, los puertos mapeados y si su estado es `healthy` o `starting`).

- `docker compose down -v`

* **Qué hace:** Detiene todos los contenedores del clúster, destruye la red virtual privada y borra cualquier volumen anónimo o caché asociada.

11.2 Inspección y Monitoreo del Motor Docker

- `docker ps --filter "name=globallogistics"`

* **Qué hace:** Lista únicamente los contenedores pertenecientes al proyecto de GlobalLogistics con su ID, imagen, comando y puertos.

- `docker stats --no-stream`

* **Qué hace:** Muestra una instantánea en tiempo real del consumo de hardware de cada contenedor (porcentaje de CPU, memoria RAM utilizada vs. límite de 512 MB, I/O de red y cantidad de procesos/PIDs).

- `docker logs globallogistics_loadbalancer`

* **Qué hace:** Muestra los registros de acceso y error de Nginx, permitiendo ver a qué IP interna de réplica reenvió cada petición entrante.

- `docker logs globallogistics_app_1`

* **Qué hace:** Muestra los logs internos de FastAPI/Uvicorn dentro de la Réplica 1 (peticiones HTTP recibidas, códigos de retorno y Health Checks periódicos).

11.3 Ejecución de Pruebas de Estrés (Apache Benchmark)

- `docker run --rm --network globallogistics_net httpd:alpine ab -n 5000 -c 100 http://globallogistics_loadbalancer/`

* **Qué hace:**

* `--rm`: Elimina el contenedor de Apache una vez que termina el benchmark para no dejar basura en el disco.

* `--network globallogistics_net`: Conecta la herramienta de prueba directamente a la red privada de los microservicios para evitar retardos externos de Windows.

* `-n 5000`: Dispara un total de 5.000 peticiones HTTP.

* `-c 100`: Mantiene 100 usuarios virtuales haciendo solicitudes de forma estrictamente simultánea (concurrencia).

- `docker run --rm --network globallogistics_net httpd:alpine ab -n 2500 -c 100 http://globallogistics_loadbalancer/stress?iterations=2500`

* **Qué hace:** Ejecuta el Escenario 2 atacando el endpoint de cómputo criptográfico intensivo con 100 usuarios concurrentes para saturar intencionalmente los 0.5 cores de CPU de cada réplica.

11.4 Scripts de Automatización y Verificación

- `powershell -ExecutionPolicy Bypass -File scripts\verify_loadbalancing.ps1`

* **Qué hace:** Envía 15 solicitudes consecutivas al balanceador, mide los milisegundos de cada una, extrae el `container_id` y `instance_name`, e imprime una tabla de distribución para demostrar el balanceo Round-Robin.

- `python scripts\run_benchmarks.py`

* **Qué hace:** Automatiza la ejecución sucesiva del Escenario 1 y Escenario 2, parsea los textos de salida de Apache Benchmark con expresiones regulares y exporta los resultados estructurados a `benchmark_results/summary.json`.

💥 12. CHAOS ENGINEERING: CÓMO SE PUEDE "ROMPER" EL SISTEMA, QUÉ PASA Y CÓMO SE MITIGA

En las defensas orales, los docentes de Arquitectura suelen hacer preguntas del tipo: *"¿Qué pasa si falla X componente?"* o *"¿Cómo harías caer esta arquitectura?"*.

A continuación se detallan los 5 escenarios reales de falla, cómo reacciona el sistema hoy y cómo se solucionaría a nivel profesional:

Escenario A: Caída de una Réplica de Microservicio (Falla de Nodo)

- **Cómo se provoca:**

`powershell

docker stop globallogistics_app_2

`

- **Qué sucede en el sistema:**

1. El contenedor `app_2` se apaga por completo.

2. Nginx intenta enviarle una petición en su turno Round-Robin y detecta que el socket TCP está cerrado (conexión rechazada).

3. **Mecanismo de Resiliencia:** Como configuramos `proxy_next_upstream error timeout`, Nginx no le devuelve un error 502 al usuario. Reintenta esa misma petición en milisegundos contra `app_1` o `app_3`.

4. Por la directiva `max_fails=3 fail_timeout=10s`, tras 3 fallos Nginx excluye a `app_2` del pool y solo reparte entre `app_1` y `app_3` (reparto 50% y 50%).

5. El usuario experimenta **Zero Downtime (cero tiempo de inactividad)**.

- **Cómo se mitiga si se caen las 3 réplicas:**

Si las 3 réplicas se apagan, Nginx devolverá `502 Bad Gateway`. La mitigación en producción es configurar una página estática de contingencia con la directiva `error_page 502 /maintenance.html;` informando al usuario que el servicio se está reiniciando.

Escenario B: Caída del Balanceador de Carga (Single Point of Failure - SPOF)

- **Cómo se provoca:**

`powershell

docker stop globallogistics_loadbalancer

`

- **Qué sucede en el sistema:**

* **Colapso total del punto de entrada:** Las 3 réplicas siguen vivas y saludables en su red privada, pero ningún cliente puede comunicarse con ellas porque el puerto `8080` deja de responder (`ERR_CONNECTION_REFUSED`).

- **Por qué ocurre:** En la arquitectura solicitada por el TP, hay exactamente **un (1) balanceador de carga**. Esto constituye un **Punto Único de Falla (SPOF)**.
- **Cómo se mitiga en la vida real (Respuesta para los profesores):**

> "En un entorno Cloud de producción real no se despliega un Nginx solitario. Se utiliza:

> 1. Un par de balanceadores Nginx en clúster activo-pasivo gestionados por **Keepalived** con una **IP Virtual Flotante (VIP) mediante VRRP**. Si el Nginx principal se apaga, el secundario toma la IP en milisegundos.

> 2. O directamente se delega la Capa 7 en un balanceador administrado por la nube como **AWS Application Load Balancer (ALB)** o **Google Cloud Load Balancing**, que cuentan con redundancia multi-zona de disponibilidad automática."

Escenario C: Ataque de Denegación de Servicio por Saturación de CPU (DDoS / Storm)

- **Cómo se provoca:**

Ejecutar Apache Benchmark con 500 usuarios concurrentes pidiendo 50.000 iteraciones de cálculo:

`powershell

docker run --rm --network globallogistics_net httpd:alpine ab -n 5000 -c 500 http://globallogistics_loadbalancer/stress?iterations=50000

`

- **Qué sucede en el sistema:**

1. Cada petición monopoliza el hilo de Python calculando hashes SHA-256 sin soltar la CPU.

2. Cada contenedor alcanza su límite de `cpus: '0.50'`. El planificador del kernel Linux suspende los procesos (*CFS Throttling*) al agotar su cuota de 50 ms por ciclo.

3. Las peticiones entrantes colapsan la cola de conexiones del socket TCP (el `backlog`).

4. Nginx alcanza el `proxy_read_timeout 15s` esperando que los microservicios le respondan y comienza a devolver errores **504 Gateway Timeout**.

- **Cómo se mitiga:**

1. **Rate Limiting en Nginx:** Configurar en `nginx.conf`:

`nginx

limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

limit_req zone=api_limit burst=20 nodelay;

`

Esto impide que un solo cliente dispare cientos de pedidos por segundo, respondiéndole `429 Too Many Requests`.

2. **Arquitectura Asíncrona con Colas:** No ejecutar tareas CPU-intensivas sincrónicamente en el endpoint HTTP. El microservicio debe recibir la solicitud, encolarla en un broker como **RabbitMQ o Apache Kafka**, responderle `202 Accepted` de inmediato al cliente, y procesar el cálculo con workers secundarios en segundo plano.

Escenario D: Fuga de Memoria (Memory Leak / OOM Kill)

- **Cómo se provoca:**

Si un desarrollador introduce un error en Python acumulando datos en una lista global sin limpiarla:

`python

leak_list = []

@app.get("/leak")

async def leak():

leak_list.append("A" * 10_000_000) # Llena 10 MB de RAM en cada llamada

`

- **Qué sucede en el sistema:**

1. La memoria del contenedor empieza a subir: 100 MB, 200 MB, 400 MB...

2. Al llegar a los `512M` definidos en el límite de `docker-compose.yml`, el demonio de Linux dispara el **Out Of Memory Killer (OOM-Killer)**.

3. El kernel mata instantáneamente el proceso de Python enviándole una señal `SIGKILL` (código de salida 137).

4. Docker Compose detecta la muerte del proceso y, por la directiva `restart: unless-stopped`, reinicia el contenedor desde cero.

- **Cómo se mitiga:**

* Mantener los límites de memoria en el manifiesto IaC para evitar que un contenedor con fuga consuma la memoria física de la máquina host y voltee a los demás servicios.

* Implementar métricas de observabilidad (Prometheus + Grafana) monitoreando el consumo de memoria mediante el endpoint `/health`.

Escenario E: Desincronización de Health Checks (Zombi / Deadlock)

- **Cómo se provoca:**

Un microservicio queda atrapado en un bucle infinito que no satura la CPU (ej. esperando un bloqueo de base de datos o un socket colgado), pero no puede responder peticiones HTTP.

- **Qué sucede en el sistema:**

1. El proceso sigue corriendo para el sistema operativo, por lo que Docker cree que el contenedor está vivo.

2. Sin embargo, el chequeo interno de salud de Docker:

`CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"`

se pasa del `timeout: 3s`.

3. Tras 3 intentos fallidos (`retries: 3`), Docker marca el contenedor con el estado **unhealthy (no saludable)**.

4. Al mismo tiempo, Nginx detecta el timeout en las peticiones reales y lo excluye mediante `max_fails=3`.

- **Cómo se mitiga:**

Configurar un **Liveness Probe** con reinicio automático: en Kubernetes, cuando un pod pasa a estado *unhealthy*, el orquestador lo mata y levanta un pod completamente nuevo y fresco de forma automática.